

Lifecycle Neglect — The Cross-Cutting Anti-Pattern Where All Three Lifecycle Events (Birth, Mating, Death) Are Treated as Implementation Details Rather Than Governed Architectural Events per B1.09, B1.10, and B1.11, Producing a Deployment Where Entities Appear, Combine, and Disappear Without Governance Anchors

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in The Instinct/Reasoning Separation Outside the Model (Li, April 2026), the second paper in the CKS theory series following Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems (Li, April 2026).

Abstract

Lifecycle Neglect is the cross-cutting anti-pattern in which all three lifecycle events — birth, mating, and death — are treated as implementation-level engineering operations rather than as governed architectural events, violating B1.09 (birth as human-governed origination), B1.10 (mating as governed cross-layer combination), and B1.11 (death as governed lifecycle closure with archival) simultaneously. The pattern is distinct from the individual lifecycle anti-patterns (B3.10 Ungoverned Birth, B3.11 Silent Death, B3.12 Untracked Mating), which each identify an isolated failure in an otherwise lifecycle-governed deployment. Lifecycle Neglect names the systematic architectural absence of lifecycle governance across the entire deployment: no lifecycle event is treated as a governance event; the architecture was never built for it. The combined effect is an entity population entirely opaque to governance — the deployment cannot enumerate what entities exist (no birth records), cannot establish how any entity originated (no mating records), and cannot determine whether absent entities were properly closed (no death records). This note formalizes the pattern under the standard B3 anti-pattern structure, identifies three emergence conditions that explain how the architecture is omitted, names four operational consequences that follow, and provides detection and remediation approaches for deployers who recognize the pattern in a live system.

1. Pattern Name

Lifecycle Neglect.

This name captures the cross-cutting character of the failure: not one lifecycle event is mishandled, but the entire lifecycle governance architecture is absent from the deployment. "Neglect" names what distinguishes this pattern from malicious omission or informed trade-off — lifecycle governance is typically omitted because the architectural requirement was not recognized, not because it was evaluated and declined. The pattern is a systematic oversight at the deployment architecture level, spanning all three lifecycle commitments.

2. Commitments Violated

Lifecycle Neglect violates three foundational lifecycle commitments from Paper 2 (Li, April 2026) simultaneously:

B1.09 — Birth as human-governed origination. Birth is the governed architectural event by which a new entity (cell, aspect, or Self) comes into existence within a CKS-governed deployment. The commitment requires that birth be recorded as a lifecycle event with provenance, that the entity's existence be anchored to an origination record in the substrate, and that humans hold authority over origination decisions. Under Lifecycle Neglect, entities are created as technical objects — class instantiations, service deployments, configuration file creations — without any governance event being recorded. Birth is an engineering act, not a governance event. The substrate carries no origination anchors for entities in the deployment.

B1.10 — Mating as governed cross-layer combination. Mating is the governed architectural event by which new entity DNA is composed from the DNA of existing entities, with three pattern variants (union, selective merge, lineage-preserved union) each producing type-appropriate results under orchestration substrate governance. The commitment requires that mating be recorded as a lifecycle event, that the resulting entity's lineage cross-reference its parent entities, and that the combination operation proceed under human-governed orchestration rules. Under Lifecycle Neglect, what would be mating in a governed deployment is performed as copy-paste engineering operations: a new entity's orchestration substrate, behavior substrate, or both are copied or derived from existing entities without any governance event being recorded. DNA similarity between entities is unexplained by any governed combination event.

B1.11 — Death as governed lifecycle closure with archival. Death is the governed architectural event by which an entity is retired from active operation, with two distinct architectural results depending on death type: functional obsolescence (genuine deletion of an entity whose function is no longer needed) and lineage supersession (retirement with archival, where the retired entity's substrate content remains substrate-addressable for lineage tracing). The commitment requires that death be recorded as a lifecycle event, that the entity's archival status be established, and that the death type be classified under governed orchestration rules. Under Lifecycle Neglect, entities are deactivated or removed as engineering cleanup operations — service shutdowns, configuration deletions, resource deallocation — without any governance event being recorded. Death is cleanup, not closure.

The three commitments are violated together, not sequentially. Lifecycle Neglect does not mean that one lifecycle event was neglected first and the others followed; it means that the deployment architecture never included lifecycle governance as an architectural concern at all.

3. Recognizable Form

Lifecycle Neglect presents as the architectural treatment of all lifecycle events as technical operations. Three dimensions of neglect are each recognizable in the deployment artifact record.

3.1 Birth-as-instantiation

The first dimension is birth treated as object instantiation or service deployment. An entity that ought to have a governed origination event — an origination record in the substrate per B1.09, a birth specification establishing the entity's purpose and initial governance configuration, a lineage anchor that downstream lifecycle events can reference — instead comes into existence as a technical act. The entity exists in the deployment from the moment it is created; it does not exist in the governance record, because no governance record was created for it.

Recognition signals: entities are present in the deployment but absent from any lifecycle record; the count of active entities in the deployment cannot be reconciled against any governed origination events; new entities can be created by engineering operations without triggering any governance workflow; entities in the deployment cannot trace their origination to a governed birth event; entity inventory from the deployment differed from entity inventory from governance records, and the difference is unexplained.

3.2 Death-as-deletion

The second dimension is death treated as deletion or cleanup. An entity that is deactivated, retired, or removed is simply deleted: its service is shut down, its configuration is removed, its resources are deallocated. No governance event is recorded. No archival determination is made. The lineage chain that would allow future entities to trace superseded ancestry is lost at deletion.

Recognition signals: entities disappear from the deployment without corresponding lifecycle closure records; entity count decreases in the deployment cannot be reconciled against governed death events; the distinction between functional-obsolescence death and lineage-supersession death is not present in the deployment's operational record; archived entities are not substrate-addressable because no archival was performed; lineage chains terminate without closure records.

3.3 Mating-as-copy

The third dimension is DNA combination treated as copy-paste engineering. When a new entity's orchestration substrate or behavior substrate is derived from existing entities — a common and entirely reasonable practice in deployment — the derivation is performed as an engineering operation. The new entity's DNA is assembled by copying, adapting, or merging content from parent entities using standard engineering tooling. No mating event is recorded.

No lineage cross-references are established. The parent entities whose DNA was used are not referenced in the new entity's governance record, because no governance record captures the combination event.

Recognition signals: new entity DNA is observably derived from existing entities (close structural or content similarity in orchestration substrates or behavior substrates) but no mating records explain the similarity; lineage cross-references are absent from entity records; the three mating pattern variants (union, selective merge, lineage-preserved union) are not present as a design concept in the deployment architecture; the provenance of any entity's DNA cannot be established from the governance record.

3.4 The combined effect

When all three dimensions are present simultaneously, the deployment's entity population is entirely opaque to governance. Governance cannot determine: what entities exist (because no birth records establish what was originated); how any entity came to be (because no mating records establish what combination events produced any entity's DNA); or whether absent entities were properly closed (because no death records establish whether entities that no longer appear in the deployment were retired through governed closure or simply deleted).

The entity population is a shadow population: it exists in engineering reality but not in governance reality. Governance has no authoritative account of what is operating, why it has the DNA it has, or whether the deployment's current entity set is complete or depleted.

4. Distinguishing Lifecycle Neglect from Individual Lifecycle Anti-Patterns

Three anti-patterns in the B3 series address individual lifecycle governance failures:

B3.10 (Ungoverned Birth) identifies a cell birth event that occurred without governed origination — one entity in an otherwise lifecycle-governed deployment was created without a birth record. The deployment has a lifecycle governance architecture; one instance of birth escaped it.

B3.11 (Silent Death) identifies a death event that occurred without governed lifecycle closure — one entity in an otherwise lifecycle-governed deployment was deleted or deactivated without a closure record. The deployment has a lifecycle governance architecture; one instance of death escaped it.

B3.12 (Untracked Mating) identifies a mating event that occurred without governed provenance — one combination operation in an otherwise lifecycle-governed deployment produced an entity without a mating record. The deployment has a lifecycle governance architecture; one instance of mating escaped it.

Each of these is a gap in an existing lifecycle governance architecture. The architecture is present; coverage is incomplete. Detection and remediation in each case focuses on the missing record for the specific event, and on tightening the governance workflow to prevent recurrence.

Lifecycle Neglect is categorically different. The lifecycle governance architecture is absent. There are no birth records to have gaps in, no death records to have gaps in, no mating records to have gaps in — because the record-keeping infrastructure was never established. The deployment's architects treated lifecycle events as engineering operations throughout. Remediation therefore cannot focus on filling gaps in an existing architecture; it must establish the lifecycle governance architecture from scratch, and then retroactively reconstruct whatever records can be reconstructed for the existing entity population.

The distinction matters for detection. A deployment exhibiting B3.10 will have birth records for most entities and one or a few entities without them. A deployment exhibiting Lifecycle Neglect will have birth records for no entities — not because records are missing, but because the concept of a birth record was not part of the deployment architecture.

5. Emergence Conditions

Three conditions explain how Lifecycle Neglect emerges in deployments that are otherwise competently designed:

Infrastructure-as-code culture. Modern AI deployment practices treat entities as deployment units managed through DevOps tooling: containers, orchestration platforms, configuration repositories, infrastructure-as-code pipelines. These tools manage entity lifecycle in the engineering sense — creation, scaling, update, teardown — with precision and auditability appropriate to their purposes. Engineers with deep fluency in DevOps tooling have a complete and sophisticated lifecycle management architecture available to them; it does not, however, include CKS lifecycle governance. Infrastructure-as-code pipelines create and destroy entities without governance integration because governance integration was not part of the pipeline design. The lifecycle is managed, but not at the governance layer. Architects working primarily in this culture may not recognize a gap, because the engineering lifecycle is comprehensively governed.

Lifecycle governance as novel concept. Paper 2's lifecycle commitment (birth, mating, death as governed primitives at every level of the CKS architecture) is an architectural commitment without a close analogue in standard software engineering or AI systems design literature. Architects familiar with software deployment lifecycle management, DevOps maturity models, and AI model lifecycle frameworks are encountering a distinct concept when they read the lifecycle governance requirements in B1.09 through B1.11. The gap between deployment lifecycle management (engineering concept, widely understood) and entity lifecycle governance (CKS architectural commitment, less widely understood) is the conceptual space in which Lifecycle Neglect emerges. The concept is novel enough that architects may read it as an organizational policy question rather than an architectural requirement, placing it outside the scope of the initial deployment architecture.

Governance conceived as operational-only. A common reading of governance in AI deployments limits governance to behavioral governance: what the system does, how it responds, what rules constrain its outputs. Under this reading, governance is the concern of the

operational layer — the rules, guardrails, and human oversight mechanisms that govern the system's behavior during operation. Entity lifecycle is then an infrastructure concern, not a governance concern, and lifecycle governance falls outside the scope of what architects build into the governance layer. This is not an unreasonable partition for many governance frameworks; it is, however, inconsistent with the CKS architecture, in which governance extends from operational behavior to entity existence. When governance is conceived as operational-only, the lifecycle governance architecture is absent by design — it was excluded from scope, not accidentally omitted.

6. Operational Consequences

Four operational consequences follow from Lifecycle Neglect:

Entity inventory impossible. Without lifecycle records anchoring entity existence to governed origination events, the deployment has no governed entity inventory. Governance cannot enumerate what entities exist and what their governance status is. The engineering record (the set of services, configurations, or containers currently running) provides a functional picture of the deployment, but that picture cannot be reconciled against any governance authority, because governance has no record of having originated the entities in that picture. Any entity in the deployment might have been created outside governed origination, and governance cannot determine which entities were and which were not — because none were.

Recursive commitments verification impossible. CKS governance commitments at the deployment level require entity inventory as a prerequisite: verification that deployment-level commitments are satisfied must begin by enumerating the entities to which those commitments apply. Without lifecycle records, entity inventory is unavailable, and verification cannot begin. This consequence is not superficial. The recursive commitments architecture is designed to allow governance to demonstrate, entity by entity and commitment by commitment, that the deployment satisfies its architecture. Lifecycle Neglect eliminates the foundation on which that demonstration is built.

Compliance architecture foundation absent. Lifecycle records are the foundational layer of the CKS compliance architecture. Birth records establish entity provenance and origination authority. Mating records establish lineage and DNA origin. Death records establish closure type and archival status. Compliance demonstrations at every level above these foundations — aspect-level compliance, Self-level compliance, evolution-mechanism compliance — depend on being able to trace claims about the deployment's current state to governed events in the entity population's history. Without the foundational lifecycle records, no compliance claim about the deployment can be made from the governance record, because the governance record does not include the deployment's entity population.

Lineage chain entirely absent. Path retraceability at entity scope — the ability to trace any entity's current DNA to a governed origination event and any intervening governed combination events — requires a lineage chain beginning at the birth record and continuing through any mating records. Without birth records or mating records, no lineage chain can be constructed.

Entities exist in the deployment without governance history. The deployment's entity population is lineage-opaque: there is no governed account of where any entity came from, whose DNA it carries, or whether that DNA was combined through governed mating events. This is the deepest consequence of Lifecycle Neglect, because lineage opacity is irreversible for entity history that predates the establishment of lifecycle governance.

7. Detection

Entity inventory reconciliation. Enumerate all entities currently present in the deployment using the engineering record (service registry, configuration repository, container inventory, or equivalent). For each entity, check for a birth record per B1.09. An entity without a birth record has ungoverned birth. If the result of this check is that no entity in the deployment has a birth record, the deployment exhibits Lifecycle Neglect rather than isolated Ungoverned Birth.

Lifecycle event audit. Examine the governance record for any lifecycle events — birth, mating, or death — recorded since deployment began. If the governance record contains no lifecycle events of any type across the deployment's history, the lifecycle governance architecture is absent. This audit can be performed independently of entity enumeration and is often faster as a first screening step: if no lifecycle events have ever been recorded, the architecture was never established.

Lifecycle verification sampling. Select a sample of entities and attempt to run lifecycle verification for each — birth verification (is there a governed origination record?), mating verification (if this entity's DNA resembles another entity's DNA, is there a mating record?), death verification (for entities that were present in earlier deployment snapshots but are no longer active, is there a closure record?). Systematic failure across all three verification types for the sampled entities confirms Lifecycle Neglect.

The distinction between Lifecycle Neglect and isolated lifecycle anti-patterns (B3.10, B3.11, B3.12) emerges cleanly in detection: isolated anti-patterns produce partial failure in entity inventory reconciliation (most entities pass, some fail); Lifecycle Neglect produces complete failure (no entities have lifecycle records of any type).

8. Remediation

Remediation for Lifecycle Neglect requires establishing the lifecycle governance architecture that was absent, then retroactively reconstructing whatever records can be reconstructed for the existing entity population.

Establish lifecycle governance architecture through directed selection. Design and deploy governance workflows that treat each lifecycle event — birth, mating, death — as a governance event requiring a record in the substrate. Birth workflows should trigger origination record creation, establish the entity's governance configuration, and produce a birth record with provenance fields populated. Death workflows should trigger lifecycle closure record creation, make the death-type determination (functional obsolescence vs. lineage supersession), and

produce archival status for the retiring entity. Mating workflows should trigger combination record creation, establish lineage cross-references from the new entity to its parent entities, and record the mating pattern used.

Retroactive birth record creation for existing entities. For entities currently in the deployment without birth records, apply the retroactive birth record creation approach: reconstruct what can be established about each entity's origination (when it was created, what authority created it, what purpose it was created for) and create a birth record capturing that reconstruction. Retroactive records should be clearly marked as reconstructed rather than contemporaneously created, with the reconstruction date and the best-available origination date both recorded. This preserves the integrity of the governance record while establishing lineage anchors for the existing entity population. Entities whose origination cannot be reconstructed should be flagged for governance review to determine whether their continued operation is consistent with the deployment's governance posture.

Establish archival procedures for future deaths. Configure death workflows to produce the type-appropriate archival result for each death type: entities retiring through lineage supersession should be archived with substrate content preserved and substrate-addressable; entities retiring through functional obsolescence may be genuinely deleted per the orchestration rules governing that death type.

Establish mating governance for DNA combinations. Configure mating workflows to trigger whenever a new entity's DNA is derived from existing entities. This requires identifying the engineering practices that currently perform mating-as-copy (template copying, configuration inheritance, substrate derivation from existing entities) and integrating governance record creation into those practices, or replacing them with governed mating operations.

Verify after establishment. After the lifecycle governance architecture is established and retroactive records are created, run lifecycle verification across the entity population to confirm that birth, mating, and death records are present for the entities that should have them, and that the entity inventory from the governance record can be reconciled against the engineering record. The presence of residual gaps after this verification identifies remaining isolated lifecycle anti-patterns (B3.10, B3.11, B3.12) that can be addressed individually through the remediation approaches for those patterns.

Source Paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to Cite This Note

Li, W. (2026). *Lifecycle Neglect — The Cross-Cutting Anti-Pattern Where All Three Lifecycle Events (Birth, Mating, Death) Are Treated as Implementation Details Rather Than Governed Architectural Events per B1.09, B1.10, and B1.11, Producing a Deployment Where Entities Appear, Combine, and Disappear Without Governance Anchors*. May 12, 2026. ORCID: 0009-0004-8065-3235.